Diseño — Panel AicoreLink: rama única, roles y menús principales

Fecha: 2026-10-14 **Estado:** borrador para aprobación de Leonardo **Documentos fuente:** `AicoreLink_Web_SRD.pdf` (ALK-SRD-001 v2.0), `AicoreLink_Web_Plan_de_Ejecucion.pdf` (ALK-PLN-001 v2.0), `AicoreLink_Web_Analisis_de_Viabilidad.pdf` **Referencia de estructura (NO de código):** `D:\OPENCODE\consultorio-medico` (AicoreMed / med.aicorebots.com) **Análisis previo:** `docs/superpowers/plans/2026-10-14-aicorelink-panel-gap-analysis.md`

---

1. Intención y por qué

El panel actual de AicoreLink no responde al SRD. El panel del cliente es una maqueta (datos hardcodeados, un no-op, un spinner infinito) y el panel interno no cumple los requisitos de prioridad Alta de M9. Además la navegación está al revés de lo que el producto necesita: no hay puerta de entrada desde la landing, no hay menú principal, y las rutas cuelgan sueltas de la raíz.

AicoreMed ya resolvió ese mismo problema en producción. **Se copia su estructura organizativa** — no su código, no su base de datos, no su IA.

El resultado buscado: un solo producto, una sola rama `/dashboard`, y dos experiencias distintas según quién se loguea.

---

2. Decisiones de Leonardo (fijas)

#	Decisión
D1	Al ingresar se va a un **menú principal** con información relevante del sistema en primera instancia
D2	**Todo el sistema deriva de una misma rama**, `/dashboard`
D3	El **login es igual al de AicoreMed**: se accede desde la página principal / landing
D4	El **rol decide el panel**: dueño de negocio → panel de cliente; alguien de Aicore → panel de Aicore
D5	El menú de Aicore muestra **información general de todos los clientes**, errores o servicios caídos, tenants y un vistazo general de cada cliente
D6	**No tocar `consultorio-medico`.** No mezclar código ni base de datos. Solo copiar la estructura
D7	La **base de datos, IA y demás servicios deben ser independientes** de AicoreMed, construidos con los repositorios que ya existen en Dokploy
D8	**Eliminar lo que no condice** con esto y construir lo correcto
D9	Libertad para que el builder tome las decisiones que falten
D10	El canal de mensajería es **Chatwoot**, no Meta Cloud API directo

---

3. Decisiones del builder (mías, revisables)

#	Decisión	Motivo
B1	Se eliminan los grupos de rutas `(cliente)` y `(admin)` y se reemplazan por **una sola rama `/dashboard`** con subcarpeta `/dashboard/admin`	Es el patrón de AicoreMed (`app/dashboard/admin/*`) y cumple D2
B2	El **layout único** de `/dashboard` resuelve la sesión, redirige a `/login` y monta el shell; el rol decide el menú y el contenido, no la ruta	Un solo lugar donde vive la autorización; el rol no se puede esquivar cambiando la URL
B3	Se **renombran los roles**: `cliente` y `dev_aicore` quedan; `admin_clinica` (basura heredada del proyecto clínico) se reemplaza por `operador` y `soporte`	El SRD RF-ADM-04 pide tres roles internos: administrador, operador de contenido, soporte solo lectura
B4	La salud se calcula con datos que **ya existen** (`TenantChannel.enabled` , `Site.status` , `PublishLog.status` , `AuditLog`) más un endpoint de health propio, sin depender de infra de AicoreMed	No hay que inventar infraestructura nueva
B5	`/fabrica` (M4) **se conserva íntegra**, solo se muda a `/dashboard/fabrica`	Es la única pieza del panel actual que está bien hecha y que el SRD pide
B6	`/whatsapp` se conserva como `/dashboard/whatsapp` , pero **se le quita el copy de clínica** ("pacientes", "staff", "La clínica abre…") y el `Cerrar sesión` duplicado	D8

#	Decisión	Motivo
B7	El panel del cliente **no** incluye `/portal` separado como en AicoreMed: el cliente de AicoreLink es el dueño del negocio, no un usuario final	El SRD no pide portal de usuario final; RF-PAN describe el panel del dueño
B8	La landing gana el link de ingreso en el header, reutilizando el `Ingresar al panel` que ya se agregó	D3, ya iniciado

4. Arquitectura de navegación objetivo

4.1 Rutas públicas (sin sesión)

Ruta	Origen
`/`	Landing `(marketing)`, con link **Ingresar** en el header → `/login`
`/login`	Se conserva (email + contraseña, NextAuth)
`/herramientas/*`, `/legal/*`	Se conservan

4.2 Rama única `/dashboard` — experiencia **cliente**

Ruta	Contenido	Requisito SRD
`/dashboard`	**Menú principal del cliente**: resumen del negocio, uso del mes vs límite del plan, leads recientes, conversaciones recientes, estado del sitio	RF-PAN-02
`/dashboard/leads`	Leads recibidos con origen (UTM) y estado	M3, RF-ANA-02
`/dashboard/conversaciones`	Conversaciones del canal + detalle con mensajes **reales**	RF-PAN-02
`/dashboard/citas`	Citas / agenda	RF-PAN-02 (hoy no existe el modelo)
`/dashboard/resenas`	Reseñas solicitadas	RF-PAN-02 (hoy no existe)
`/dashboard/sitio`	Ver el sitio, editarlos datos del negocio y publicar el cambio	RF-PAN-03
`/dashboard/reportes`	Reporte mensual (consultas atendidas, citas, reseñas, fuera de horario, ingresos recuperados)	RF-PAN-04
`/dashboard/soporte`	Solicitudes de cambio/soporte con estado	RF-PAN-05
`/dashboard/mi-cuenta`	Datos del negocio, plan, facturación	M10

4.3 Rama única `/dashboard` — experiencia **Aicore** (`/dashboard/admin`)

Ruta	Contenido	Requisito SRD
`/dashboard` (rol Aicore)	**Menú principal de Aicore**: salud global, alertas/errores, tenants en riesgo y vistazo por cliente	RF-ADM-01, D5
`/dashboard/admin/tenants`	Lista con plan, estado de pago, uso, salud del sitio y del canal, y marcadores de riesgo	RF-ADM-01
`/dashboard/admin/tenants/[id]`	Detalle por cliente	RF-ADM-01
`/dashboard/admin/onboarding`	Asistente de incorporación con checklist de 12–15 pasos por cliente	RF-ADM-03
`/dashboard/admin/fabrica`	Fábrica de sitios (movida desde `/fabrica`)	M4
`/dashboard/admin/fabrica/[siteId]`	Panel de revisión y publicación	M4
`/dashboard/admin/embudo`	Embudo **alimentado por datos reales** (visitas → leads → calificados → citas → clientes)	RF-ANA-04
`/dashboard/admin/metricas`	Métricas y atribución, exportación CSV	M11, RF-ANA-05
`/dashboard/admin/infraestructura`	Servicios, salud, `audit_log`	RF-ADM-05
`/dashboard/admin/usuarios`	Roles internos	RF-ADM-04

Regla dura: las rutas `/dashboard/admin/*` verifican el rol **en el servidor** (layout o guard). Un `cliente` que escriba la URL recibe 403/redirect. Sin excepciones.

---

5. Modelo de roles y control de acceso

`TenantUser.role` hoy: `cliente` | `admin_clinica` | `dev_aicore`.

Roles objetivo:

Rol	Quién	Alcance
`cliente`	Dueño del negocio	Ve solo su tenant
`dev_aicore`	Administrador de Aicore	Ve todo; puede publicar, cambiar plan, exportar, borrar
`operador`	Operador de contenido	Ve todo; edita y publica contenido; no cambia plan ni borra
`soporte`	Soporte	Solo lectura

- `multiTenant` (ya existe en la sesión JWT) sigue siendo la marca de acceso interno.
- El aislamiento por fila sigue siendo RLS con `current_setting('app.current_tenant')`; el panel de Aicore usa el rol de aplicación con `BYPASSRLS` para poder leer cross-tenant (ya es el patrón de la fábrica).

---

6. Menú principal — experiencia Aicore (lo que pidió Leonardo)

Orden de bloques, de arriba hacia abajo:

1. **Tira de salud del sistema** — Postgres, n8n, Ollama/IA, entrega de WhatsApp, publicación de sitios. Cada servicio: OK / degradado / caído, con la última verificación.
2. **Alertas activas** — errores agrupados: publicaciones fallidas (`PublishLog.status = failed`), canales deshabilitados, tenants sin uso en 7 días, tenants sobre el límite, pagos vencidos. Cada alerta con severidad y a qué cliente afecta.
3. **Totales del negocio** — clientes activos, MRR, leads del mes, conversaciones atendidas, sitios publicados.
4. **Tabla "vistazo por cliente"** — una fila por tenant: nombre, plan, estado, uso del mes vs límite, salud del sitio, salud del canal, último mensaje, y semáforo de riesgo. Ordenada por riesgo descendente.

Todo con datos reales; ningún número hardcodeado.

---

7. Menú principal — experiencia cliente

1. **Encabezado del negocio** — nombre del tenant (no `Clínica Demo`), rubro, plan.
2. **Uso del mes vs límite** — conversaciones, citas, minutos de IA, con el límite del plan visible.
3. **Leads recientes** — últimos ingresos con su origen.
4. **Conversaciones recientes** — con acceso al detalle y mensajes reales.
5. **Estado del sitio** — publicado / en revisión / sin publicar, con acceso a `/dashboard/sitio` .
6. **Accesos rápidos** — pedir un cambio, ver el reporte del mes.

---

8. Salud y alertas — de dónde salen los datos

Señal	Fuente existente
Canal de WhatsApp activo	`TenantChannel.enabled`
Sitio publicado / borrador	`Site.status`
Publicación fallida	`PublishLog.status = failed` , `error`
Actividad del cliente	`Conversation.updatedAt` , `Message.createdAt`
Uso del mes	`TenantUsage`

Señal	Fuente existente
Costo y margen	`CostRate`, `TenantUsage`
Acciones sensibles	`AuditLog`
Servicios de infraestructura	Endpoint de health propio que consulta Postgres, n8n y Ollama

Lo único nuevo es el endpoint de health. Todo lo demás ya está en la base.

---

9. Infraestructura — independencia de AicoreMed

Recurso	AicoreMed	AicoreLink
Base de datos	`consultorio_medico`	**`aicorelink`** (ya existe, separada)
Usuario de BD	`dashboard_user`	`aicorelink_app` (ya existe)
IA	Ollama + Whisper + ElevenLabs del stack médico	**Instancia/colección propia** de AicoreLink
App	`med.aicorebots.com`	`aicorelink.aicorebots.com`
Repositorio Dokploy	propio	**propio** (`aicorelink-aicorelink-b7m7ol`)
n8n	workflows médicos	workflows de AicoreLink (webhook de leads ya existe)
Mensajería	—	**Chatwoot propio** (instancia de AicoreLink, independiente)

Regla: ningún recurso, tabla, credencial ni workflow se comparte con `consultorio-medico`. Se copia el patrón de despliegue, nunca los datos ni los identificadores.

---

10. Qué se elimina (D8)

Elemento	Acción
`Clínica Demo`, `Pro` y el estado hardcodeados en `(cliente)/dashboard/page.tsx`	**Eliminado**
Comparación sin sentido `"activo" === "activo"`	**Eliminado**
`loadMessages()` no-op	**Reemplazado** por carga real de mensajes
Spinner infinito en estados vacíos	**Reemplazado** por estado vacío real
Doble navegación (`<nav>` propio dentro de la página)	**Eliminado** — queda solo el shell
`Cerrar sesión` duplicado en `/whatsapp`	**Eliminado**
Locale `es-AR`	**Reemplazado** por `es-CL`
Copy clínico: "pacientes", "staff", "La clínica abre…"	**Eliminado**
Rol `admin_clinica`	**Reemplazado**
Grupos de rutas `(cliente)` y `(admin)` en la raíz	**Eliminados**
`/metricas` inaccesible (disabled sin link)	**Integrada** en `/dashboard/admin/metricas`
`/embudo` con etapas constantes	**Reemplazado** por datos reales

---

11. Fases

Fase	Contenido	Resultado
F0	Restructura: una rama, roles, layout único, link de ingreso en la landing, redirección por rol	El esqueleto correcto
F1	Menú principal de Aicore: salud, alertas, totales, vistazo por cliente	Cumple D5 y RF-ADM-01

Fase	Contenido	Resultado
F2	Menú principal de cliente: uso vs límite, leads, conversaciones con mensajes reales, estado del sitio	Cumple RF-PAN-02
F3	`/dashboard/sitio` : edición de datos del negocio y publicación	Cumple RF-PAN-03
F4	Soporte con estado y reporte mensual	Cumple RF-PAN-04 y RF-PAN-05
F5	Onboarding con checklist y roles internos	Cumple RF-ADM-03 y RF-ADM-04
F6	Embudo y métricas con datos reales, exportación CSV	Cumple M11
F7	Citas, reseñas, facturación	Requiere decisiones de negocio pendientes

Cada fase cierra con los gates del proyecto: `lint` , `typecheck` , `vitest` , `build` .

---

12. No-objetivos

- No se toca `consultorio-medico` .
- No se comparten base de datos, credenciales ni workflows con AicoreMed.
- No se implementa portal de usuario final (B7).
- No se implementa pasarela de pago en este diseño: depende de una decisión de negocio (RF-BIL-07).
- No se rediseña la landing: ya pasó su rediseño visual.

---

13. Respuestas al builder check (2026-10-14)

Un revisor independiente leyó este documento y devolvió seis dudas. Resolución de cada una:

#	Duda	Resolución
1	¿El rol interno es global o por vínculo tenant-usuario?	**Decisión del builder.** `TenantUser.role` es por vínculo y `lib/auth.ts` rechaza el login si el usuario no tiene ningún tenant, así que el staff de Aicore no podría entrar. Se agrega un **rol de staff en `User`** (global). `TenantUser.role` queda para lo del cliente (`cliente` , `admin_cliente`). `multiTenant` NO significa "es staff": hoy es `tenantLinks.length > 1` y se deja de usar para autorizar
2	¿RLS o BYPASSRLS? ¿Una conexión o dos?	**Decisión del builder, con verificación.** `asAdmin()` abre una transacción y el comentario de `lib/tenant-context.ts` asume un rol con `BYPASSRLS` . Con una sola conexión, darle `BYPASSRLS` al rol de la app **anularía el RLS para el cliente también**. Se usan **dos conexiones**: `DATABASE_URL` (con RLS, para el cliente) y `DATABASE_URL_ADMIN` (rol con `BYPASSRLS` , solo para lecturas cross-tenant del panel de Aicore). Hay que verificar en la VPS si `aicorelink_app` ya tiene `BYPASSRLS`
3	¿Señales de facturación (MRR, pagos vencidos)?	**Fuera de alcance por ahora.** No existe ningún modelo de facturación: `Tenant` solo tiene `plan` y `status` . F1 muestra plan, uso y actividad; las señales de pago entran cuando exista M10
4	¿Existen límites numéricos por plan?	**Respondido por el SRD, sección 3.2 "Uso justo".** Conversaciones IA/mes: 300 / 1.500 / 5.000. Minutos de voz: — / — / 120. Plantillas WhatsApp: — / 300 / 1.000. Cambios de contenido: 1 / 3 / 8. Y **RC-04 (Alta)**: avisar al cliente y al operador al **80% y al 100%** del uso justo
5	¿Destinos reales del health?	**Decisión del builder + descubrimiento en la VPS.** El endpoint consulta Postgres, n8n y la instancia de IA propia (D7). La tira NO depende de infraestructura de AicoreMed
6	¿Hay conversaciones reales en producción?	**Hay que verificar en la VPS.** Si las tablas `conversations` / `messages` están vacías, F2 necesita una fase previa de verificación de la ingesta antes de mostrar "mensajes reales"

---

14. Decisiones que quedan abiertas para Leonardo

1. ~~**Canal de WhatsApp**~~ → **RESUELTO (D10): Chatwoot.** Chatwoot es la fuente de verdad de las conversaciones: la bandeja del cliente, M6 y la tira de salud leen de Chatwoot. Queda un trabajo derivado: reconciliar el webhook actual `app/api/webhooks/whatsapp/route.ts` (que habla Meta Cloud API) y el widget muerto `components/marketing/chatwoard-widget.tsx` , y decidir si la integración es por webhook de Chatwoot o por su API.
2. **RF-PAN-01**: ¿acceso del cliente con código por WhatsApp / enlace mágico, o seguimos con email y contraseña?
3. **RF-BIL-07**: ¿qué pasarela chilena (Flow, Mercado Pago, Transbank) y entra la facturación en este esfuerzo o queda para más adelante?

4. **Citas y reseñas**: ¿se implementan de verdad (requiere modelos nuevos y agenda) o se muestran como "próximamente" hasta la fase 7?